Business Profile System - Developer Quick Reference

Entity Relationships

```
User (1) ──→ (N) BusinessProfile
              │
              ├──→ (N) BankDetails
              └──→ (N) VerificationLog
```

Core Concepts

1. Multi-Tenancy Pattern

Every business operation MUST be scoped by `business_profile_id`:

```
// ❌ WRONG - No tenant isolation
List<Order> orders = orderRepository.findAll();

// ☑ CORRECT - Scoped by business
Long businessId = activeBusinessProfileContext.getActiveBusinessProfileId();
List<Order> orders = orderRepository.findByBusinessProfileId(businessId);
```

2. Active Business Context

Always use `ActiveBusinessProfileContext` in session:

```
@Autowired
private ActiveBusinessProfileContext context;

// After login, let user pick business
context.setActiveBusinessProfile(profileId, businessName, userId);

// In all queries
if (!context.hasActiveBusinessProfile()) {
    throw new BusinessNotSelectedException();
}
Long activeBusinessId = context.getActiveBusinessProfileId();
```

3. Repository Query Pattern

Always validate user ownership:

```
// ❌ WRONG - Can access any business
BusinessProfile bp = businessProfileRepository.findById(id).orElseThrow();

// ☑ CORRECT - Validates user owns the business
Long userId = getCurrentUserId();
BusinessProfile bp = businessProfileRepository
    .findByIdAndUserId(id, userId)
    .orElseThrow(() -> new UnauthorizedException());
```

Common Operations

Create Business Profile

```
BusinessProfile profile = new BusinessProfile();
profile.setUser(currentUser);
profile.setLegalBusinessName("ABC Pvt Ltd");
profile.setBusinessType(BusinessType.PRIVATE_LIMITED);
profile.setGstin("29ABCDE1234F1Z5");
profile.setPanNumber("ABCDE1234F"); // Auto-encrypted
profile.setRegisteredAddress("123 Main St");
profile.setState("Karnataka");
profile.setPincode("560001");
profile.setVerificationStatus(VerificationStatus.DRAFT);

businessProfileRepository.save(profile);
```

Add Bank Account

```
BankDetails bank = new BankDetails();
bank.setBusinessProfile(businessProfile);
bank.setAccountHolderName("ABC Private Limited");
bank.setAccountNumber("1234567890123456"); // Auto-encrypted
bank.setIfscCode("SBIN0001234");
bank.setBankName("State Bank of India");
bank.setPrimary(true); // First account is primary

bankDetailsRepository.save(bank);
```

Log Verification Attempt

```
VerificationLog log = new VerificationLog();
log.setBusinessProfile(businessProfile);
log.setVerificationType(VerificationType.GST);
log.setVerificationProvider("GSTN API");
log.setRequestPayload(sanitizedRequestJson);
```

```
log.setResponsePayload(sanitizedResponseJson);
log.setVerificationResult("SUCCESS");
log.setIpAddress(request.getRemoteAddr());
log.setUserAgent(request.getHeader("User-Agent"));

verificationLogRepository.save(log);
```

Get Primary Bank Account (Settlement)

```
BankDetails primaryAccount = bankDetailsRepository
    .findPrimaryVerifiedAccount(businessProfileId)
    .orElseThrow(() -> new NoPrimaryAccountException());

// Use for payment settlement
processSettlement(primaryAccount);
```

Validation Examples

GST Number Validation

```
// Pattern: 29ABCDE1234F1Z5 (15 chars)
@Pattern(regexp = "^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$")
private String gstin;
```

PAN Number Validation

```
// Pattern: ABCDE1234F (10 chars)
@Pattern(regexp = "^[A-Z]{5}[0-9]{4}[A-Z]{1}$")
private String panNumber;
```

IFSC Code Validation

```
// Pattern: SBIN0001234 (11 chars)
@Pattern(regexp = "^[A-Z]{4}0[A-Z0-9]{6}$")
private String ifscCode;
```

Security Checklist for Every Query

☑ **Validate User Ownership**

```
```

```
// Always check user_id
businessProfileRepository.findByIdAndUserId(id, userId);
```

☑ **Scope by Active Business**

```
// Always filter by business_profile_id
Long businessId = context.getActiveBusinessProfileId();
orderRepository.findByBusinessProfileId(businessId);
```

☑ **Never Expose Encrypted Data**

```
// ❌ WRONG - Exposes encrypted account number
log.info("Account: " + bank.getAccountNumber());

// ☑ CORRECT - Use masked version
log.info("Account: " + bank.getMaskedAccountNumber());
```

☑ **Validate Before Sensitive Operations**

```
if (!context.validateUser(currentUserId)) {
    throw new UnauthorizedException();
}
```

Enum Usage

Business Type

```
businessProfile.setBusinessType(BusinessType.PRIVATE_LIMITED);
String display = businessProfile.getBusinessType().getDisplayName();
```

Verification Status

```
businessProfile.setVerificationStatus(VerificationStatus.VERIFIED);
if (businessProfile.getVerificationStatus().canTransact()) {
    // Allow transactions
}
```

Bank Verification

```
bank.setBankVerificationStatus(BankVerificationStatus.VERIFIED);
if (bank.canUseForSettlement()) {
    // Use for settlements
}
```

Common Queries

Find User's Businesses

```
List<BusinessProfile> businesses =
    businessProfileRepository.findByUserId(userId);
```

Find Active Businesses

```
List<BusinessProfile> activeBusinesses =
    businessProfileRepository.findActiveProfilesByUserId(userId);
```

Check GSTIN Exists

```
if (businessProfileRepository.existsByGstin(gstin)) {
    throw new DuplicateGstinException();
}
```

Find Recent Verifications

```
LocalDateTime sevenDaysAgo = LocalDateTime.now().minusDays(7);
List<VerificationLog> recentLogs = verificationLogRepository
    .findRecentLogs(businessProfileId, sevenDaysAgo);
```

API Integration Pattern

Verification Flow

```
public void verifyGST(Long businessProfileId, String gstin) {
    try {
        // Call external API
        GSTResponse response = gstApiClient.verify(gstin);

        // Log attempt
        VerificationLog log = new VerificationLog();
        log.setBusinessProfile(getBusinessProfile(businessProfileId));
```

```
        log.setVerificationType(VerificationType.GST);
        log.setVerificationResult(response.isValid() ? "SUCCESS" : "FAILED");
        log.setRequestPayload(sanitize(request));
        log.setResponsePayload(sanitize(response));
        verificationLogRepository.save(log);

        // Update business profile
        if (response.isValid()) {
            businessProfile.setVerificationStatus(VerificationStatus.VERIFIED);
            businessProfileRepository.save(businessProfile);
        }
    } catch (Exception e) {
        // Log error
        logVerificationError(businessProfileId, VerificationType.GST, e);
    }
}
```

Error Handling

Custom Exceptions

```
public class BusinessNotSelectedException extends RuntimeException {
    public BusinessNotSelectedException() {
        super("No business profile selected");
    }
}

public class NoPrimaryAccountException extends RuntimeException {
    public NoPrimaryAccountException() {
        super("No primary verified bank account found");
    }
}

public class UnauthorizedBusinessAccessException extends RuntimeException {
    public UnauthorizedBusinessAccessException() {
        super("You don't have access to this business profile");
    }
}
```

Testing Templates

Unit Test

```
@Test
public void testBusinessProfileCreation() {
    User user = new User("testuser", "test@example.com", "password");

    BusinessProfile profile = new BusinessProfile();
    profile.setUser(user);
```

```
    profile.setLegalBusinessName("Test Business");
    profile.setBusinessType(BusinessType.PROPRIETORSHIP);
    profile.setGstin("29ABCDE1234F1Z5");
    profile.setPanNumber("ABCDE1234F");

    BusinessProfile saved = businessProfileRepository.save(profile);
    assertNotNull(saved.getId());
    assertEquals(VerificationStatus.DRAFT, saved.getVerificationStatus());
}
```

Integration Test

```
@Test
public void testTenantIsolation() {
    User user1 = createUser("user1");
    User user2 = createUser("user2");

    BusinessProfile bp1 = createBusinessProfile(user1);
    BusinessProfile bp2 = createBusinessProfile(user2);

    // User1 should not access User2's business
    Optional<BusinessProfile> result =
        businessProfileRepository.findByIdAndUserId(bp2.getId(), user1.getId());

    assertFalse(result.isPresent());
}
```

Performance Tips

1. Use Indexes

Queries automatically use indexes on:

- `user_id`
- `gstin`
- `pan_number`
- `business_profile_id`
- `verification_status`

2. Fetch Strategy

```
// For lists (avoid N+1)
@Query("SELECT bp FROM BusinessProfile bp " +
       "LEFT JOIN FETCH bp.bankAccounts " +
       "WHERE bp.user.id = :userId")
List<BusinessProfile> findByUserIdWithBanks(@Param("userId") Long userId);
```

3. Pagination

```
Pageable pageable = PageRequest.of(0, 20);
Page<VerificationLog> logs =
    verificationLogRepository.findByBusinessProfileId(businessProfileId,
pageable);
```

Troubleshooting

Issue: Encryption Error

Problem: "Decryption failed"
Solution: Check ENCRYPTION_SECRET_KEY environment variable

Issue: No Primary Account

Problem: Cannot process settlement
Solution: At least one bank account must have `is_primary=true`

Issue: Cross-Business Data

Problem: Seeing data from other businesses
Solution: Always use `findByIdAndUserId()` or scope by `business_profile_id`

Issue: Flyway Migration Failed

Problem: Schema validation error
Solution:

```
# Reset Flyway (development only)
DELETE FROM flyway_schema_history;
```

Constants for Reference

```
// GSTIN Format: 2-digit state + 5-letter PAN + 4-digit number + letter + checksum
+ Z + alphanumeric
// Example: 29ABCDE1234F1Z5

// PAN Format: 5 letters + 4 digits + 1 letter
// Example: ABCDE1234F

// CIN Format: 1 letter (U/L) + 5 digits + 2 letters + 4 digits + 3 letters + 6
digits
// Example: U12345MH2020PTC123456

// Udyam Format: UDYAM-2letter-2digit-7digit
// Example: UDYAM-KA-01-0000123
```

```
// IFSC Format: 4 letters + 0 + 6 alphanumeric
// Example: SBIN0001234
```

Useful Queries for Debugging

```sql
-- Check user's businesses
SELECT * FROM business_profiles WHERE user_id = ?;

-- Check primary accounts
SELECT * FROM bank_details WHERE business_profile_id = ? AND is_primary = true;

-- Check verification history
SELECT * FROM verification_logs
WHERE business_profile_id = ?
ORDER BY created_at DESC LIMIT 10;

-- Find businesses without primary bank
SELECT bp.id, bp.legal_business_name
FROM business_profiles bp
LEFT JOIN bank_details bd ON bp.id = bd.business_profile_id AND bd.is_primary =
true
WHERE bd.id IS NULL;
```

Remember

- **Security**: Always validate user ownership
- **Tenant Isolation**: Always scope by business_profile_id
- **Encryption**: PAN/Account numbers are auto-encrypted
- **Audit**: Log all verification attempts
- **Validation**: Use entity validators before save
- **No Cross-Access**: Never expose other user's businesses